

Types of Data Structures

1. **Linear Data Structures**

Store data sequentially

include: arrays, linked lists, stacks queues

2. **Non-linear data structures**

Store data in a hierarchical manner

include: graphs, trees

Linked Lists

A linked list is a linear data structure made up of a collection of elements that are linked with each other, and each element points to the next one.

These elements are better known as **nodes**.

These nodes are not stored at contiguous location, but linked using pointers to a different memory location.

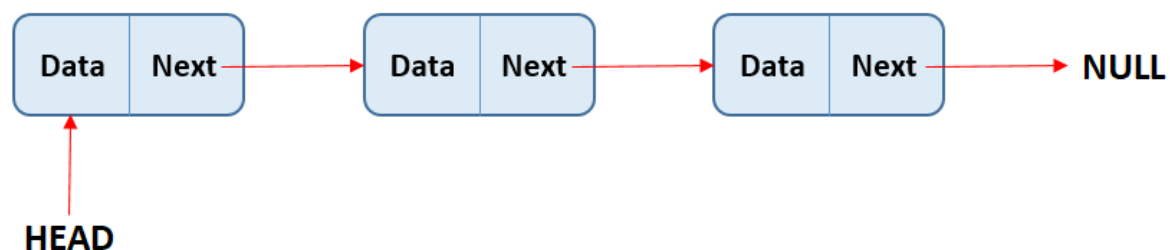
Each node in a linked list contains two items:

- Data/value(like a name or number)
- Reference to the next element/pointer

The first node in a linked list is called a **Head**.

The last node in a linked list points to null/empty value.

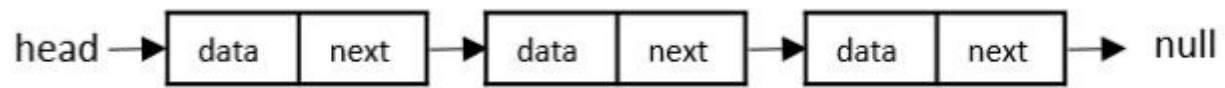
Linked lists are a **dynamic** data structure since it does not have a given size and amount of memory pre-defined. They can grow easily in size and memory.



Types of Linked Lists

- **Singly linked list** - one described above; with the first element(head) pointing to the next element and the last element pointing to null.

- Traversal can only be done in one direction. This is because each node only knows about the next node and not the node before.



```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class SinglyLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def insert(self, data):
```

```
        new_node = Node(data)
```

```
        if not self.head:
```

```
            self.head = new_node
```

```
            return
```

```
        temp = self.head
```

```
        while temp.next:
```

```
            temp = temp.next
```

```
        temp.next = new_node
```

```

def traverse(self):

    temp = self.head

    while temp:

        print(temp.data, end=" -> ")

        temp = temp.next

    print("None")

```

- **Doubly linked list** - each node has two references; one to the next element and another to the previous element.
 - Traversal is done twice which means, can traverse linearly and also in the reverse order.



```

class Node:

```

```

    def __init__(self, data):

        self.data = data

        self.prev = None

        self.next = None

```

```

class DoublyLinkedList:

```

```

    def __init__(self):

        self.head = None

```

```

    def insert(self, data):

```

```
new_node = Node(data)

if self.head is None:

    self.head = new_node

    return

temp = self.head

while temp.next:

    temp = temp.next

temp.next = new_node

new_node.prev = temp


def traverse_forward(self):

    temp = self.head

    while temp:

        print(temp.data, end=" <-> ")

        temp = temp.next

    print("None")


def traverse_backward(self):

    temp = self.head

    while temp.next:

        temp = temp.next

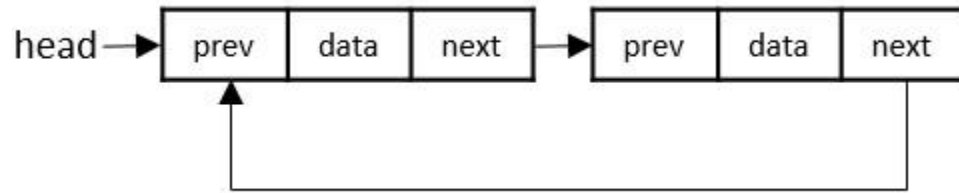
    while temp:

        print(temp.data, end=" <-> ")

        temp = temp.prev
```

```
print("None")
```

- **Circular linked list** - where the last node points back to the first node.
 - Can exist for both singly linked lists and doubly linked lists
 - Since the last node and first node are connected the traversal can go on forever until broken



```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class CircularLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def insert(self, data):
```

```
        new_node = Node(data)
```

```
        if not self.head:
```

```
            self.head = new_node
```

```
            new_node.next = self.head
```

```

        return

temp = self.head
while temp.next != self.head:
    temp = temp.next
temp.next = new_node
new_node.next = self.head

def traverse(self):
    if not self.head:
        print("List is empty")
        return
    temp = self.head
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.head:
            break
    print("(back to head)")

```

Basic Operation in Linked Lists

1. Insertion Operation

Basic procedure of insertion at the beginning:

- START
- Create a node to store the data
- Check if the list is empty

- If the list is empty, add the data to the node and assign the head pointer to it.
- If the list is not empty, add the data to a node and link to the current head. Assign the head to the newly added node.
- END

```
def insert_at_beginning(self, data):
```

```
    new_node = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new_node
```

```
    else:
```

```
        new_node.next = self.head
```

```
        self.head.prev = new_node
```

```
        self.head = new_node
```

Basic procedure of insertion at the end:

- Start
- Create a new node and assign the data
- Find the last node
- Point the last node to the new node
- End

```
def insert_at_end(self, data):
```

```
    new_node = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new_node
```

```
    return
```

```
    temp = self.head
```

```
    while temp.next:
```

```
temp = temp.next  
temp.next = new_node  
new_node.prev = temp
```

Basic procedure of insertion at a given position:

- Start
- Create a new node and assign data to it
- Iterate until the node at position is found
- Point first to new first node
- End

```
def insert_at_position(self, data, position):
```

```
    new_node = Node(data)
```

```
    if position == 0:
```

```
        self.insert_at_beginning(data)
```

```
    return
```

```
    temp = self.head
```

```
    for _ in range(position - 1):
```

```
        if temp is None:
```

```
            return
```

```
        temp = temp.next
```

```
    if temp is None:
```

```
        return
```

```
    new_node.next = temp.next
```

```
    if temp.next:
```

```
        temp.next.prev = new_node
```

```
    temp.next = new_node
```



```
new_node.prev = temp
```

2. Deletion Operation

Basic procedure of Deletion at the beginning:

- Start
- Assign the head pointer to the next node in the list
- End

```
def delete_at_beginning(self):
```

```
    if self.head is None:
```

```
        return
```

```
    self.head = self.head.next
```

```
    if self.head:
```

```
        self.head.prev = None
```

Basic procedure of Deletion at the end:

- Start
- Iterate until you find the second last element in the list
- Assign Null to the second last element in the list
- End

```
def delete_at_end(self):
```

```
    if self.head is None:
```

```
        return
```

```
    temp = self.head
```

```
    while temp.next:
```

```
        temp = temp.next
```

```
    if temp.prev:
```

```
temp.prev.next = None  
else:  
    self.head = None
```

Basic procedure of insertion at a given position:

- Start
- Iterate until find the current node at position in the list
- Assign the adjacent node of current node in the list to its previous node
- End

```
def delete_at_position(self, position):
```

```
    if self.head is None:
```

```
        return
```

```
    temp = self.head
```

```
    for _ in range(position):
```

```
        if temp is None:
```

```
            return
```

```
        temp = temp.next
```

```
    if temp is None:
```

```
        return
```

```
    if temp.prev:
```

```
        temp.prev.next = temp.next
```

```
    if temp.next:
```

```
        temp.next.prev = temp.prev
```

3. Reverse Operation

- START
- We use three pointers to perform the reversing: prev, next, head.
- Point the current node to head and assign its next value to the prev node.
- Iteratively repeat step 3 for all the nodes in the list.
- Assign head to the prev node.

```
def reverse(self):
```

```
    temp = None
```

```
    current = self.head
```

```
    while current:
```

```
        temp = current.prev
```

```
        current.prev = current.next
```

```
        current.next = temp
```

```
        current = current.prev
```

```
    if temp:
```

```
        self.head = temp.prev
```

4. Search Operation

- START
- If the list is not empty, iteratively check if the list contains the key
- If the key element is not present in the list, unsuccessful search
- END

```
def search(self, key):
```

```
    temp = self.head
```

```
    while temp:
```

```
    if temp.data == key:
        return True
    temp = temp.next
return False
```

5. Traversal Operation

- Start
- While the list is not empty and did not reach the end of the List, print the data in each node
- End

```
def display(self):
    temp = self.head
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.next
    print("None")
```

Linked lists are great for **adding** and **removing** elements. However, they are slow to search and access elements.

In comparison to arrays, linked lists are great for inserting and deleting elements while arrays are great for random access and traversing through the elements.